

Manual técnico del Agente RAG

Vicente Rivas Monferrer & Juan M. Alberola

jalberola@dsic.upv.es

Grado en Tecnologías Interactivas

Antes de empezar

Este documento es complementario al enunciado de la práctica. Asume que ya lo habéis leído y que tenéis claro qué se os pide, qué bandas hay, cuál es el contrato de interfaz que tiene que cumplir `consultar.py` y cómo se evalúa.

Aquí cubrimos las dos cosas que el enunciado deja fuera a propósito: **cómo dejar el entorno listo** (Parte I) y **cómo construir el agente por dentro** (Parte II), incluyendo el salto de single-agent a hexagonal si aspiráis al 10. La guía vale para Linux, macOS y Windows.

Qué es y qué no es este manual

Es: una receta práctica para que arranquéis sin pelearos con el entorno y un mapa del código por dentro. Las decisiones de chunking, k , prompt y arquitectura siguen siendo vuestras.
No es: una rúbrica. La rúbrica está en el enunciado. Si algo de aquí entra en conflicto con el enunciado, manda el enunciado.

Repositorio base: <https://github.com/vjrivmon/agente-rag-gti>. Trae los 16 `.txt` del corpus DNI, un `consultar.py` de banda 5 funcional y las plantillas vacías (`features.json`, `GRUPO.md`, `AI_USAGE.md`).

Parte I — Instalación del entorno

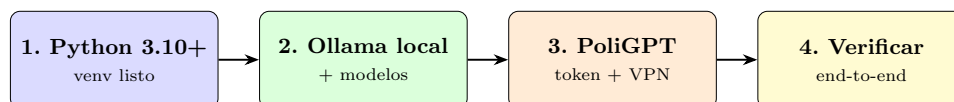


Figura 1: Flujo de instalación. Los cuatro pasos en orden: Python, Ollama, PoliGPT, verificación. AWS Rekognition se trata en el Anexo A.

1. Python 3.10 o superior

Linux/macOS:

```
python3 --version          # comprobar versión instalada
sudo apt install python3.11 python3.11-venv  # Ubuntu/Debian
```

```
brew install python@3.11 # macOS Homebrew
```

Windows: descargad el instalador desde <https://www.python.org/downloads/>, marcando *Add Python to PATH*.

Crear un entorno virtual para la práctica desde la raíz del repo clonado:

```
python3 -m venv .venv
source .venv/bin/activate # Linux/macOS
.venv\Scripts\activate # Windows
pip install -r requirements.txt
```

2. Ollama y modelos locales

2.1 Instalación de Ollama

Linux:

```
curl -fsSL https://ollama.com/install.sh | sh
```

macOS y Windows: descargad el binario desde <https://ollama.com/download>.
Verificad que funciona:

```
ollama --version
ollama serve & # arranca el servicio en segundo plano
```

Por defecto Ollama escucha en <http://localhost:11434>.

2.2 Matriz de modelos pequeños recomendados

Para banda 7 hay que comparar **4 modelos** (2 PoliGPT + 2 locales). Esta es la matriz validada para portátiles de gama media-baja:

Modelos locales para portátiles de gama media-baja

Modelo Ollama	Tamaño	RAM mín	Español	Notas
llama3.2:1b	1.3 GB	4 GB	Aceptable	Mínimo absoluto. Itera rápido.
llama3.2:3b	2.0 GB	6 GB	Bueno	Sweet spot calidad/peso.
qwen2.5:3b	1.9 GB	6 GB	Muy bueno	Fuerte en multilingüe.
gemma3:4b	3.3 GB	6 GB	Muy bueno	Calidad alta para su tamaño.
gemma2:2b	1.6 GB	4 GB	Bueno	Eficiente en CPU pura.

Y un modelo de **embeddings** (no entra en la comparativa de banda 7, pero lo necesitáis siempre):

```
ollama pull nomic-embed-text # 274 MB, 768 dims, multilingüe
```

2.3 Descargar modelos

```
ollama pull llama3.2:3b
ollama pull qwen2.5:3b
ollama pull gemma3:4b
ollama pull nomic-embed-text
```

Cada pull tarda entre 1 y 5 minutos según vuestra conexión. Verificad con:

```
ollama list
```

2.4 Probar el LLM

```
ollama run llama3.2:3b "¿Qué es Damos Nuestra Ilusión?"
```

El modelo no conoce DNI, así que dará una respuesta genérica o directamente inventada. Esto es **esperado** y motiva por qué necesitáis RAG.

3. PoliGPT

PoliGPT es la API de modelos LLM de la UPV. Es compatible con la API de OpenAI, así que cualquier cliente OpenAI vale.

3.1 Acceso

- Endpoint: <https://api.poligpt.upv.es/v1>.
- **VPN UPV**: obligatoria desde fuera del campus. Sin VPN, las llamadas a PoliGPT dan timeout silencioso. La guía oficial del ASIC para configurar la VPN está en <https://www.upv.es/contenidos/INFOACCESO/>.
- Catálogo de modelos: cambia con el tiempo. Antes de fijar los dos que usaréis en el benchmark, haced GET <https://api.poligpt.upv.es/v1/models> con vuestra clave API proporcionada para ver qué hay disponible *ahora*.

3.2 Configurar la clave

Nunca subáis la clave al repositorio. Cargadla como variable de entorno o en un archivo `.env` excluido de git:

```
# .env
POLIGPT_API_KEY=<vuestro_token>
POLIGPT_BASE_URL=https://api.poligpt.upv.es/v1
```

3.3 Probar PoliGPT

```
import os
from openai import OpenAI

client = OpenAI(
    base_url=os.environ["POLIGPT_BASE_URL"],
    api_key=os.environ["POLIGPT_API_KEY"],
)

resp = client.chat.completions.create(
    model="poligpt",
    messages=[{"role": "user", "content": "Hola"}],
    temperature=0.2,
)

print(resp.choices[0].message.content)
```

4. Verificación end-to-end

Antes de empezar a desarrollar, comprobad que toda la cadena funciona:

```
# 1. Ollama responde
curl http://localhost:11434/api/tags

# 2. Modelo carga
ollama run llama3.2:3b "Hola"

# 3. Embeddings funcionan
curl http://localhost:11434/api/embeddings -d '{
  "model": "nomic-embed-text",
  "prompt": "test"
}'

# 4. ChromaDB importable
python -c "import chromadb; print(chromadb.__version__)"

# 5. PoliGPT responde (con VPN UPV activa)
python -c "import os; from openai import OpenAI; \
  c=OpenAI(base_url=os.environ['POLIGPT_BASE_URL'], \
    api_key=os.environ['POLIGPT_API_KEY']); \
  print(c.models.list().data[0].id)"
```

Si los 5 pasos pasan, ya podéis empezar a tocar código.

Parte II — Construcción del agente

5. Recursos del repositorio base

Ficheros que vais a tocar:

- `base_conocimiento/` — 16 documentos `.txt` del corpus DNI. **No los modifiquéis.**
- `consultar.py` — punto de entrada del agente. El repo trae un esqueleto banda 5 que ya cumple el contrato del enunciado.
- `requirements.txt` — dependencias mínimas.
- `features.json`, `GRUPO.md`, `AI_USAGE.md` — plantillas que rellenáis al entregar.

Dependencias mínimas sugeridas:

```
requests
chromadb
langchain-text-splitters
sentence-transformers # opcional, alternativa a Ollama embeddings
rank-bm25             # opcional, retrieval híbrido
ragas                 # banda 8
boto3                 # opcional, AWS Rekognition (extra)
```

6. Estructura del corpus

Cada documento del corpus tiene un patrón distinto: el FAQ es Q:/A:, las presentaciones son prosa larga, los listados de horarios son tablas. **No forcéis todo a un único formato:** ese desorden controlado es parte del realismo del caso DNI.

6.1 Recomendaciones de chunking

- Empezad con `RecursiveCharacterTextSplitter` de `langchain-text-splitters` con `chunk_size=500` y `chunk_overlap=100`. Es lo que se usa en clase.
- Verificad cuántos chunks se generan por documento. Documentos como `15_desayunos_100_preguntas.txt` producirán muchos; documentos como `04_filosofia_dni.txt` pocos.
- Cuidad los Q:/A: del FAQ: si un chunk corta entre la pregunta y su respuesta, el retrieval pierde efectividad. Considerad pre-procesar agrupando cada par.
- **Conservad el nombre del archivo origen** en los metadatos del chunk. Es necesario para citar fuentes (banda 6+).

6.2 Embeddings recomendados

- `nomic-embed-text` vía Ollama (768 dims, multilingüe, sin red externa).
- `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` (384 dims, rápido, sin Ollama).

6.3 Vector store

ChromaDB en disco (`persistent_client`) es la opción más sencilla. FAISS también es válido pero requiere serializar manualmente. En cualquier caso, guardad junto al embedding al menos:

```
{
  "id": "01_faq_dni__chunk_07",
  "embedding": [...],
  "text": "<chunk literal>",
  "metadata": {
    "source": "01_faq_dni.txt",
    "chunk_index": 7
  }
}
```

7. Esqueleto banda 5

Este es el código que ya viene en `consultar.py` del repositorio base. Lo dejamos aquí como referencia visual; al clonar el repo lo tenéis ejecutable directamente.

```
from pathlib import Path
import requests
import chromadb
from langchain_text_splitters import RecursiveCharacterTextSplitter

OLLAMA_URL = "http://localhost:11434/api" # Ollama local (banda 5)
LLM_MODEL = "gemma2:27b"
```

```

EMBED_MODEL = "nomic-embed-text"

# 1) Cargar corpus
docs = []
for path in sorted(Path("base_conocimiento").glob("*.txt")):
    docs.append({"name": path.name, "text": path.read_text(encoding="utf-8")})

# 2) Chunking
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
chunks = []
for doc in docs:
    for i, c in enumerate(splitter.split_text(doc["text"])):
        chunks.append({"id": f"{doc['name']}_{i}", "text": c,
                       "source": doc["name"]})

# 3) Embeddings + indexar
def embed(text):
    r = requests.post(f"{OLLAMA_URL}/embeddings",
                      json={"model": EMBED_MODEL, "prompt": text})
    return r.json()["embedding"]

client = chromadb.Client()
col = client.get_or_create_collection("dni")
for ch in chunks:
    col.add(ids=[ch["id"]], embeddings=[embed(ch["text"])],
            documents=[ch["text"]], metadatas=[{"source": ch["source"]}])

# 4) Consulta
def consultar(pregunta, conversation_id=None):
    q_emb = embed(pregunta)
    res = col.query(query_embeddings=[q_emb], n_results=5)
    contexto = "\n\n".join(res["documents"][0])
    prompt = f"""\n\nEres un asistente de la asociación DNI. Responde
    SOLO usando el contexto. Si no está, di que no lo sabes.

    CONTEXTO:
    {contexto}

    PREGUNTA: {pregunta}
    RESPUESTA: """""
    r = requests.post(f"{OLLAMA_URL}/generate",
                      json={"model": LLM_MODEL, "prompt": prompt,
                            "stream": False,
                            "options": {"temperature": 0.2}})
    return {"respuesta": r.json()["response"],
            "fuentes": [m["source"] for m in res["metadatas"][0]]}

```

Es **intencionadamente plano**: todo en un único archivo, sin clases, sin separar concerns. Es la línea de partida. Para banda 10 hay que refactorizarlo a la arquitectura que describimos a continuación.

8. Arquitectura hexagonal (banda 10)

La **arquitectura hexagonal**, también conocida como *ports & adapters*, fue propuesta por Alistair Cockburn en 2005. La idea central es simple: **el dominio de vuestra aplicación no debe depender de detalles externos** (qué LLM usáis, qué vector store, qué framework HTTP). Esos detalles se conectan al dominio mediante *ports* (interfaces) y *adapters* (implementaciones).

8.1 Por qué hacerlo

- Cambiar de Ollama a PoliGPT se reduce a escribir un nuevo *adapter*, sin tocar el dominio.
- Probar el dominio con un LLM *fake* (que devuelve respuestas pre-grabadas) es trivial.
- Si en el futuro queréis cambiar ChromaDB por Qdrant, el dominio sigue intacto.

8.2 Tres capas concéntricas

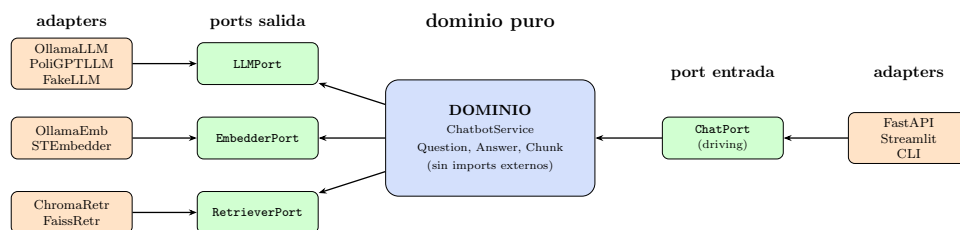


Figura 2: Arquitectura hexagonal aplicada al Asistente DNI. El dominio es agnóstico al LLM, al vector store y a la UI. Los *adapters* (naranja) son intercambiables sin tocar el dominio.

8.3 Estructura de carpetas propuesta

```

agente-rag-gti/
|-- domain/
|   |-- entities.py          # Question, Answer, Chunk (dataclasses)
|   |-- chatbot.service.py   # lógica del RAG (no importa nada externo)
|   \-- ports.py            # LLMPort, EmbedderPort, RetrieverPort, ChatPort
|-- adapters/
|   |-- llm/
|   |   |-- ollama_llm.py
|   |   |-- poligpt_llm.py
|   |   \-- fake_llm.py
|   |-- embedder/
|   |   |-- ollama_embedder.py
|   |   \-- st_embedder.py
|   |-- retriever/
|   |   |-- chroma_retriever.py
|   |   \-- faiss_retriever.py
|   \-- web/
|       |-- fastapi_app.py
|       |-- cli.py
|       \-- streamlit_ui.py
|-- config.py                # composition root: monta los adapters

```

```
\-- tests/  
  \-- test_chatbot_service.py  # usa FakeLLM, FakeRetriever
```

8.4 Ejemplo: definición de un *port* y un *adapter*

Port (domain/ports.py):

```
from typing import Protocol  
  
class LLMPort(Protocol):  
    def generate(self, prompt: str, *, temperature: float = 0.2) -> str: ...  
  
class RetrieverPort(Protocol):  
    def retrieve(self, query: str, *, k: int = 5) -> list[Chunk]: ...
```

Adapter (adapters/llm/ollama_llm.py):

```
import requests  
from domain.ports import LLMPort  
  
class OllamaLLM: # implementa LLMPort estructuralmente  
    def __init__(self, base_url: str, model: str):  
        self.base_url = base_url  
        self.model = model  
  
    def generate(self, prompt: str, *, temperature: float = 0.2) -> str:  
        r = requests.post(f"{self.base_url}/generate", json={  
            "model": self.model, "prompt": prompt,  
            "stream": False, "options": {"temperature": temperature}  
        })  
        return r.json()["response"]
```

Servicio de dominio (domain/chatbot_service.py):

```
from domain.ports import LLMPort, RetrieverPort  
from domain.entities import Question, Answer  
  
class ChatbotService:  
    def __init__(self, llm: LLMPort, retriever: RetrieverPort):  
        self.llm = llm  
        self.retriever = retriever  
  
    def answer(self, question: Question) -> Answer:  
        chunks = self.retriever.retrieve(question.text, k=5)  
        context = "\n\n".join(c.text for c in chunks)  
        prompt = self._build_prompt(question.text, context)  
        text = self.llm.generate(prompt)  
        return Answer(text=text, sources=[c.source for c in chunks],  
                      chunks=chunks)  
  
    def _build_prompt(self, q: str, ctx: str) -> str:  
        return f"...PROMPT BIEN DISEÑADO AQUÍ...\nCTX:\n{ctx}\nQ: {q}"
```


Composition root (config.py):

```
from adapters.llm.ollama_llm import OllamaLLM
from adapters.retriever.chroma_retriever import ChromaRetriever
from domain.chatbot_service import ChatbotService

def build_chatbot():
    llm = OllamaLLM(base_url="http://localhost:11434/api", model="gemma2:27b")
    retriever = ChromaRetriever(collection_name="dni")
    return ChatbotService(llm=llm, retriever=retriever)
```

8.5 Test del dominio sin LLM real

Con esta arquitectura el test del dominio es trivial y rápido:

```
class FakeLLM:
    def generate(self, prompt, *, temperature=0.2):
        return "Los desayunos son a las 8h (fuente: 01_faq_dni.txt)"

class FakeRetriever:
    def retrieve(self, query, *, k=5):
        return [Chunk(source="01_faq_dni.txt",
                       text="Q: ¿Hora desayunos? A: 8h",
                       score=0.95)]

def test_chatbot_returns_answer_with_source():
    bot = ChatbotService(FakeLLM(), FakeRetriever())
    answer = bot.answer(Question(text="¿A qué hora son los desayunos?"))
    assert "8" in answer.text
    assert "01_faq_dni.txt" in answer.sources
```

8.6 Recursos para profundizar**Lectura imprescindible (gratis):**

- Alistair Cockburn, *Hexagonal Architecture* (paper original): <https://alistair.cockburn.us/hexagonal-architecture/>.
- Bob Gregory & Harry Percival, *Architecture Patterns with Python* (a.k.a. “Cosmic Python”), libro completo gratuito en <https://www.cosmicpython.com/>. Capítulos 4–6 cubren *ports* & *adapters* en Python.

Vídeos:

- ArjanCodes, *Hexagonal Architecture: What You Need to Know* en YouTube: explicación visual con ejemplos en Python.
- Tom Hombergs (autor de *Get Your Hands Dirty on Clean Architecture*), charla *Clean Architecture with Java* en YouTube. El lenguaje no importa, los principios son idénticos.

Artículos en español:

- CodelyTV, blog y curso introductorio sobre arquitectura hexagonal en castellano (<https://codely.com/>).

9. Recomendaciones generales

- Versionad desde el primer commit. Commits pequeños y legibles.
- Medid antes de optimizar: si el chunking da malos resultados, sabed qué preguntas concretas fallan antes de tocar parámetros.
- Cuidad el *prompt*: gran parte de la calidad final viene de aquí, no del modelo.
- Si una respuesta del LLM es mala, mirad primero los chunks recuperados. Si los chunks son malos, el LLM no puede salvar la respuesta.

Anexo — AWS Rekognition (extra +1.5)

Para el extra de Rekognition (subís una foto de horarios y el asistente la coteja con los horarios oficiales del corpus):

```
pip install boto3
```

Cargad la clave AWS como variable de entorno (nunca al repo):

```
# Linux/macOS
export AWS_ACCESS_KEY_ID="<tu_clave>"
export AWS_SECRET_ACCESS_KEY="<tu_secret>"

# Windows PowerShell
$env:AWS_ACCESS_KEY_ID="<tu_clave>"
$env:AWS_SECRET_ACCESS_KEY="<tu_secret>"
```

Script mínimo:

```
import boto3
client = boto3.client('rekognition', region_name='us-west-2')

with open("foto_horarios.jpg", "rb") as f:
    img_bytes = f.read()

response = client.detect_text(Image={'Bytes': img_bytes})
texto_extraido = " ".join(d['DetectedText']
                           for d in response['TextDetections']
                           if d['Type'] == 'LINE')

print(texto_extraido)
```

A partir de aquí, el texto extraído se pasa al asistente DNI como pregunta o como contexto adicional, comparando con el corpus oficial. El extra se valora por la **pertinencia de la integración**, no por usar Rekognition de forma desconectada.